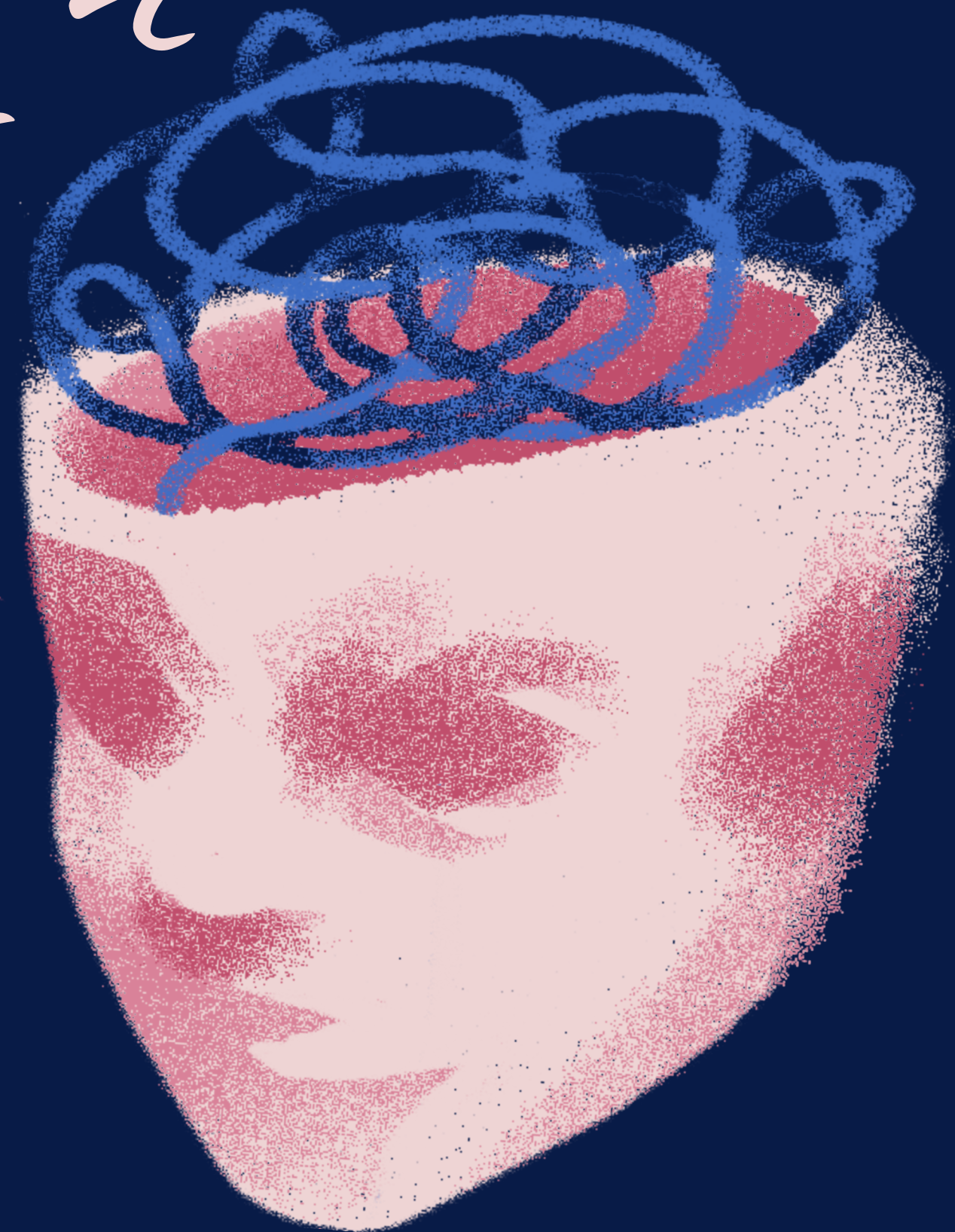


Histogram Equalization

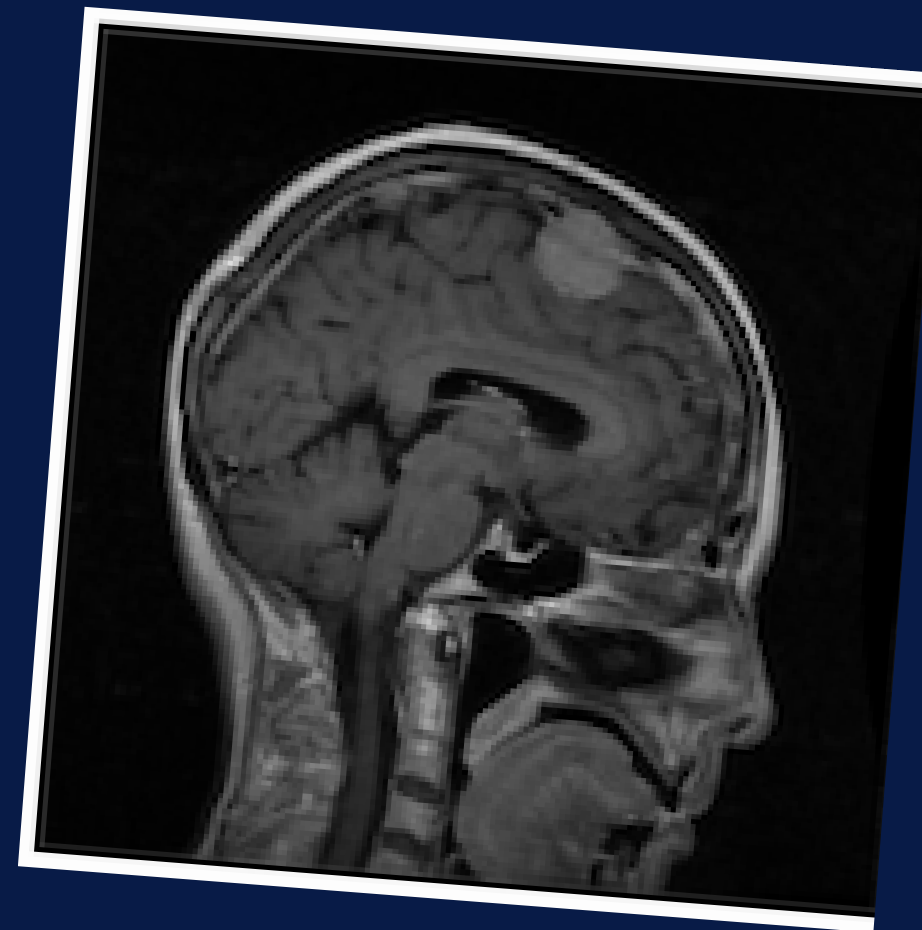
Brain Tumor MRI

Parallel Computing
with CUDA



♥ Thapaht jirameharat

Why it's CRUCIAL?

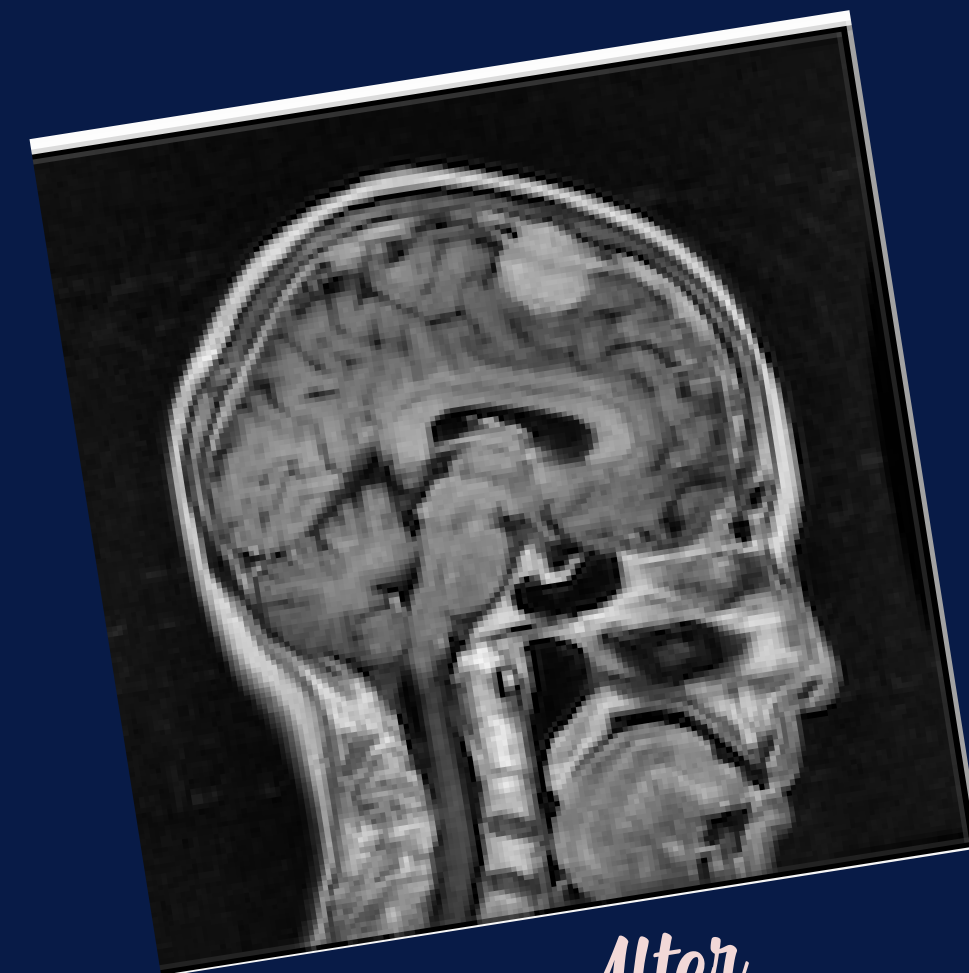


Before



เน้นย้ำความผิดปกติ

ภาพ MRI ทางการแพทย์มักมีช่วงความสว่าง (INTENSITY RANGE) แคบ ทำให้ความแตกต่างระหว่างเนื้อเยื่อปกติกับเนื้องอกไม่เด่นชัด HE จะ กระจายค่าความสว่างของพิกเซลให้สม่ำเสมอทั่วทั้งช่วง (เช่น 0-255) ซึ่งช่วย เน้นขอบ และ รายละเอียด ของเนื้องอกให้เห็นได้ง่ายขึ้นมากสำหรับแพทย์ และใช้เทรน MODEL CLASSIFICATION ต่อ



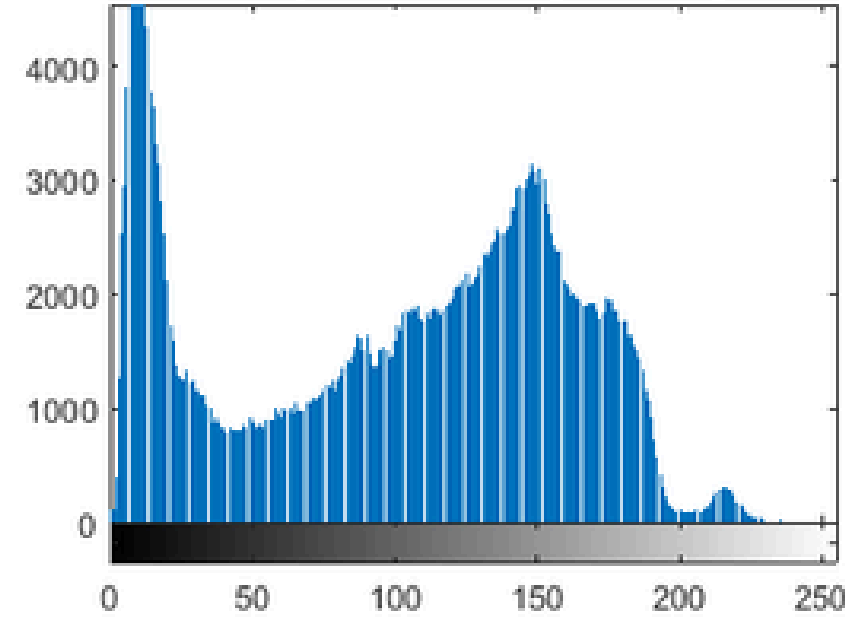
After

Histogram Equalization

Original Image



Histogram of an Original image



Before

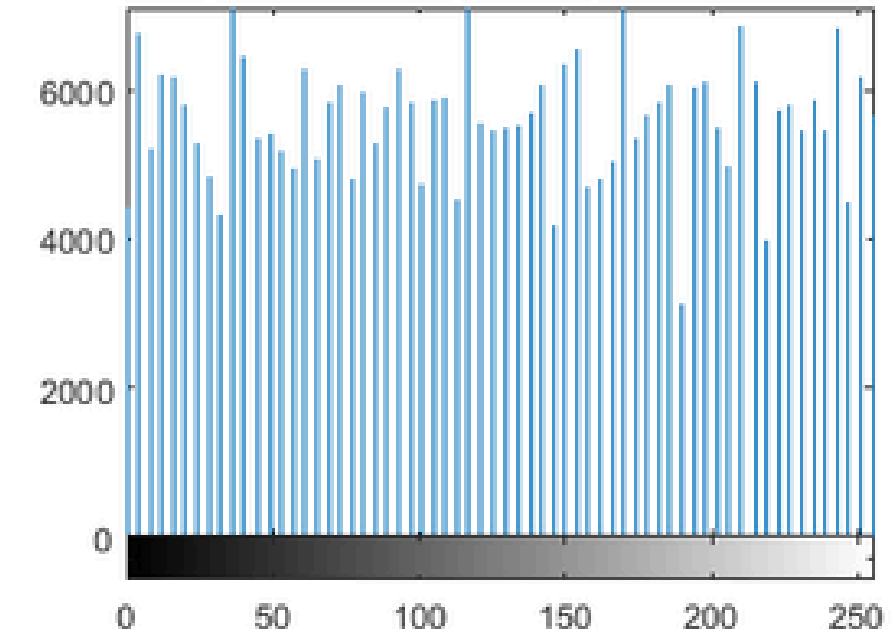


After

Histogram Equalization Image



Histogram of Histogram Equalization Image



Histogram Equalization

$$\text{PDF}[i] = \frac{h[i]}{N}$$

$$\text{CDF}[i] = \sum_{j=0}^i \text{PDF}[j]$$

$$(L - 1) \times \text{CDF}[i]$$

4x4 = 16 pixel MN = 16
DEPTH (L) = 8
L-1 = 7

1	2	1	1
2	0	1	3
2	0	1	2
1	0	2	3

ค่าสีเดิม (rk)	จำนวนพิกเซล (nk)	ความน่าจะเป็น (PDF)	ความน่าจะเป็นสะสม (CDF)	คำนวณค่าใหม่	ค่าสีใหม่ (sk)
0	3	3/16=0.1875	0.1875	7×0.1875=1.3125	1
1	6	6/16=0.3750	0.1875+0.3750=0.5625	7×0.5625=3.9375	4
2	5	5/16=0.3125	0.5625+0.3125=0.8750	7×0.8750=6.1250	6
3	2	2/16=0.1250	0.8750+0.1250=1.0000	7×1.0000=7.0000	7
4	0	0	1	7	7
5	0	0	1	7	7
6	0	0	1	7	7
7	0	0	1	7	7

Histogram Equalization

0 → 1

1 → 4

2 → 6

3 → 7

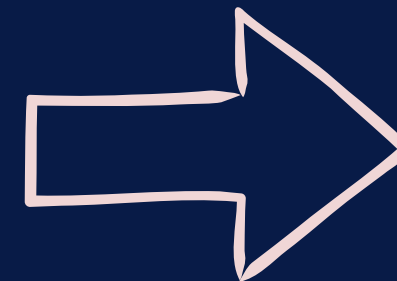
4 → 7

5 → 7

6 → 7

7 → 7

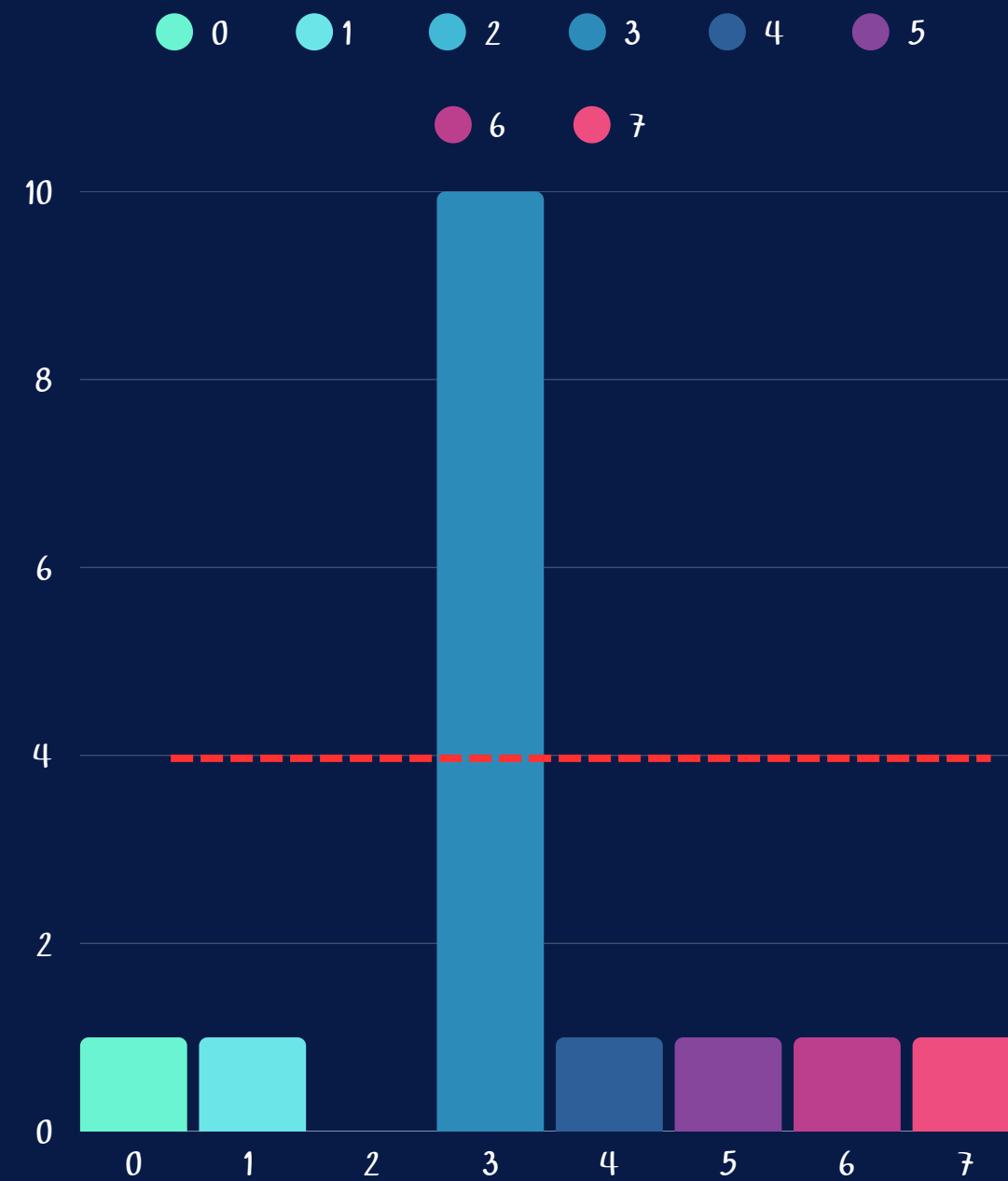
1	2	1	1
2	0	1	3
2	0	1	2
1	0	2	3



4	6	4	4
6	1	4	7
6	1	4	6
4	1	6	7

CLaHE

Clip Limit (β) = 4



ค่าสี (r_k)	จำนวนเดิม (n_k)	เช็คขีดจำกัด (Max 4)	ส่วนเกิน (Excess)	จำนวนหลังตัด (h_{clip})
0	1	ผ่าน	0	1
1	1	ผ่าน	0	1
2	0	ผ่าน	0	0
3	10	เกิน!	$10 - 4 = 6$	4
4	1	ผ่าน	0	1
5	1	ผ่าน	0	1
6	1	ผ่าน	0	1
7	1	ผ่าน	0	1

CLAFHE



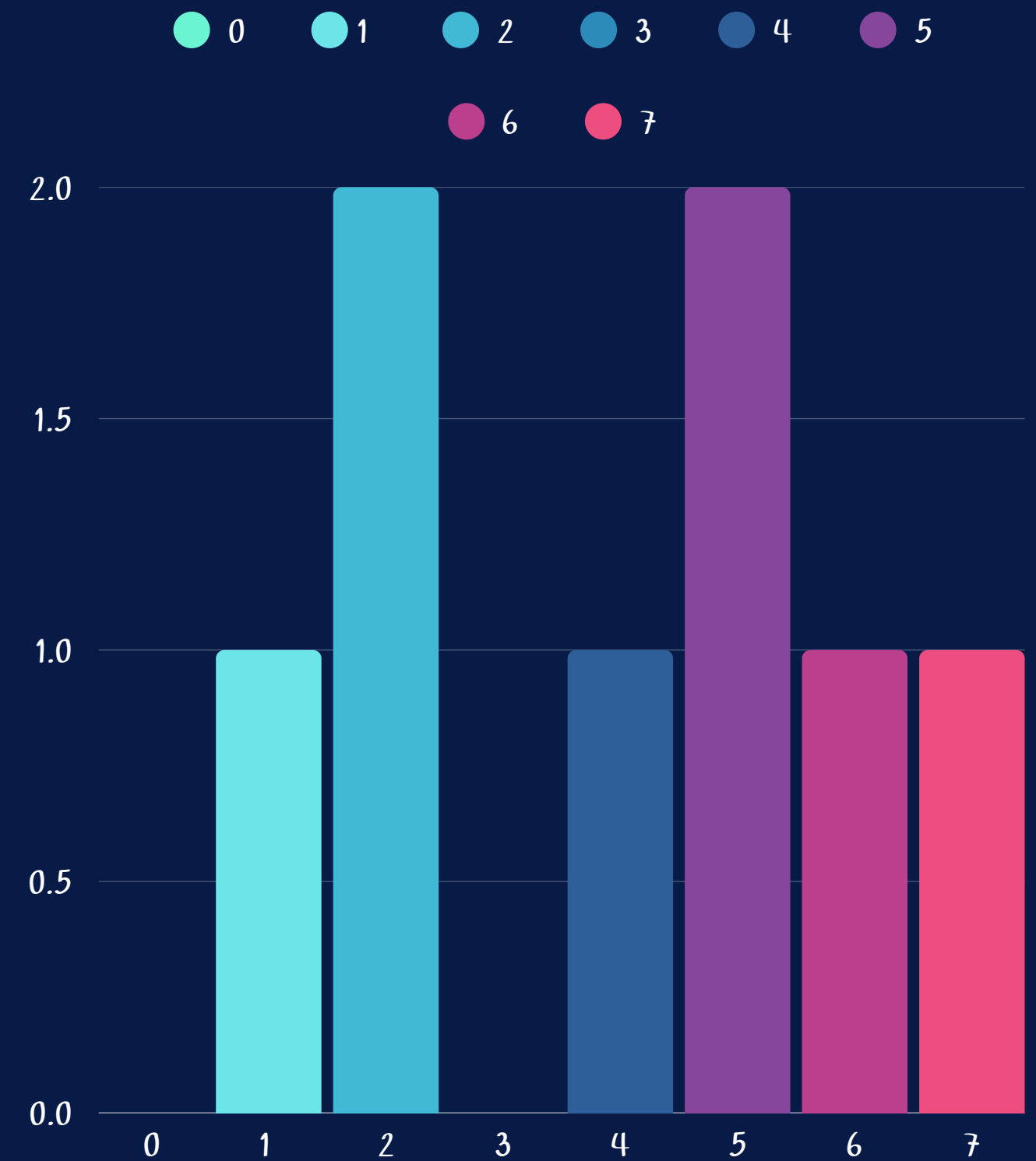
บวกคั่น

ค่าสี	h_{clip}	บวกคั่น (+0.75)	จำนวนสุทธิ (h_{final})
0	1	+0.75	1.75
1	1	+0.75	1.75
2	0	+0.75	0.75
3	4	+0.75	4.75
4	1	+0.75	1.75
5	1	+0.75	1.75
6	1	+0.75	1.75
7	1	+0.75	1.75

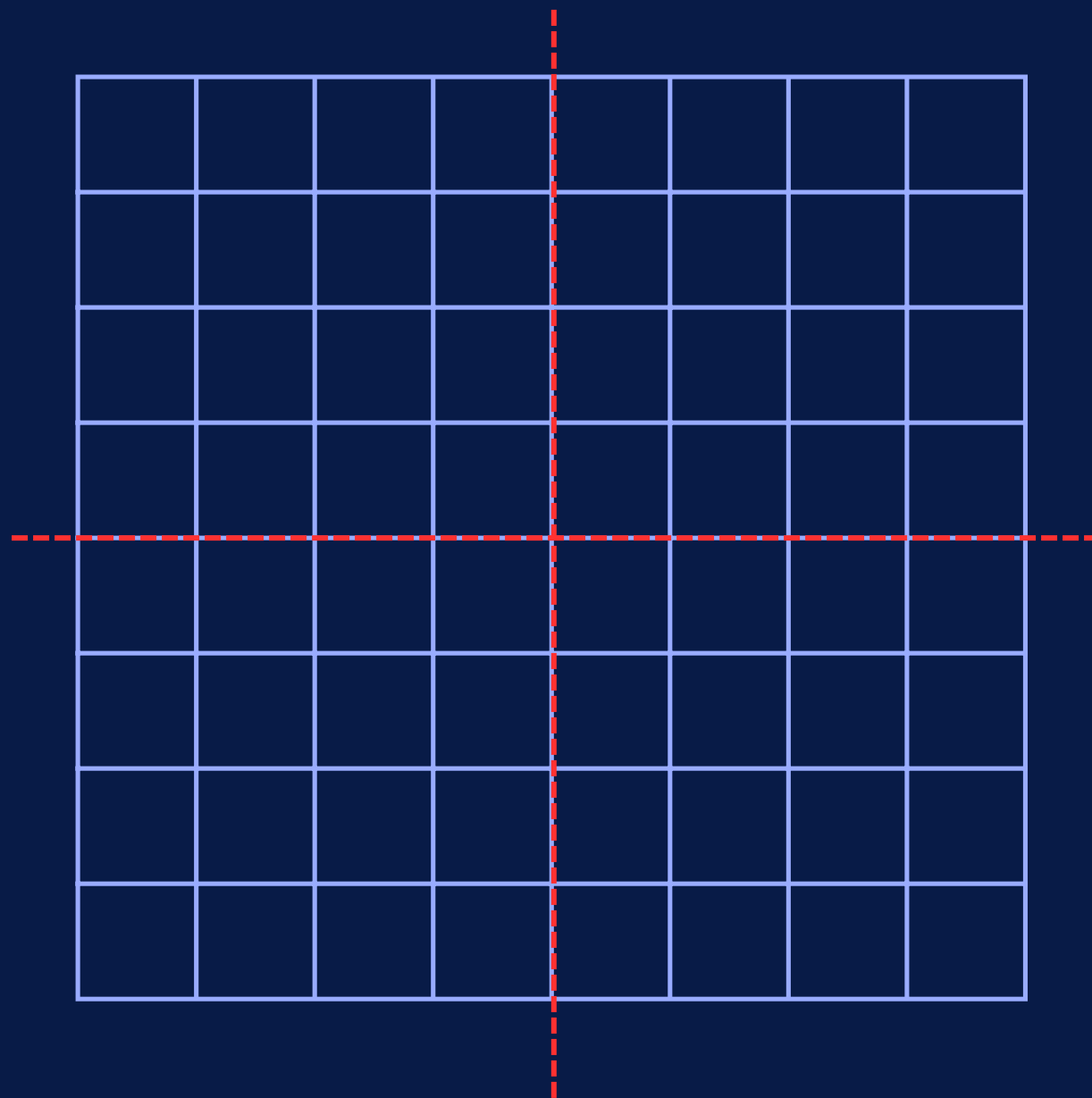
$$\frac{\text{Excess}}{\text{Total Bins}} = \frac{6}{8} = \mathbf{0.75}$$

CLaHE

ค่าสี (r_k)	h_{final}	PDF ($h/16$)	CDF (สะสม)	$CDF \times 7$	ค่าใหม่ (s_k)	เทียบ HE ปกติ*
0	1.75	0.109	0.109	0.76	1	0
1	1.75	0.109	0.218	1.52	2	1
2	0.75	0.047	0.265	1.85	2	1
3	4.75	0.297	0.562	3.93	4	7 (โดดไปสุด)
4	1.75	0.109	0.671	4.70	5	7
5	1.75	0.109	0.780	5.46	5	7
6	1.75	0.109	0.889	6.22	6	7
7	1.75	0.109	1.000	7.00	7	7



CLAH



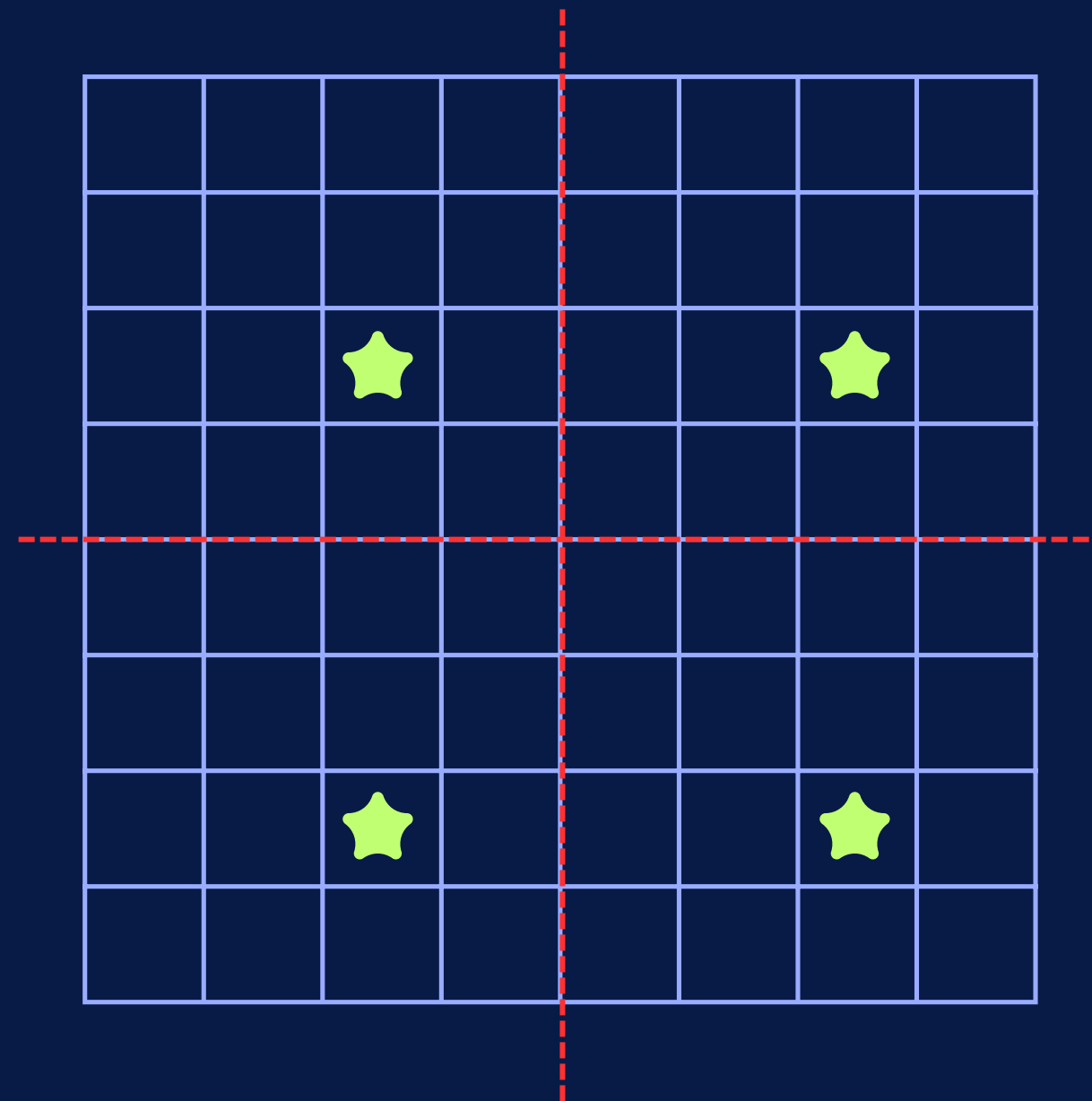
Tile

CLASSE

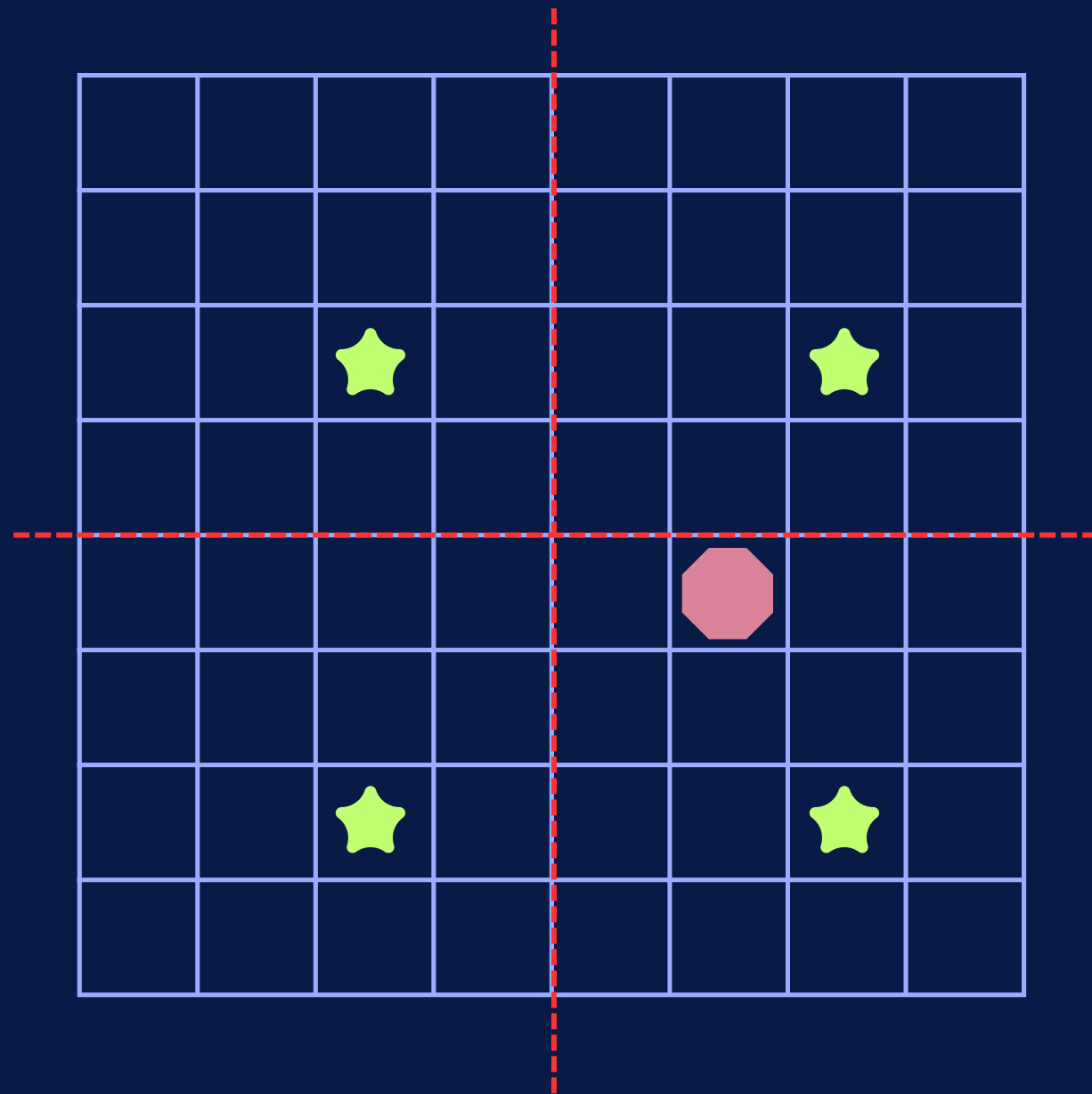


CLAH

$$\text{Center} = \text{จุดเริ่มต้นของ Tile} + \left(\frac{\text{ขนาด Tile}}{2}\right)$$



CLATE



พิกัด (4, 5)

แนวนอน (แกน X): อยู่ที่หลัก 5

ห่างจากซ้าย (2) = 3 หน่วย

ห่างจากขวา (6) = 1 หน่วย

คำนวณน้ำหนัก : $\frac{\text{ระยะจากซ้าย}}{\text{ระยะรวม}} \rightarrow a = 0.75$ $\frac{\text{ระยะจากขวา}}{\text{ระยะรวม}} \rightarrow a-1 = 0.25$

แนวตั้ง (แกน Y): อยู่ที่แถว 4

ห่างจากบน (2) = 2 หน่วย

ห่างจากล่าง (6) = 2 หน่วย

อยู่กึ่งกลางพอดี ($b = 0.5$) $b = 0.5$, $b-1 = 0.5$

$$Val = b \times [(a-1) \times TL) + (a \times TR)] + b-1 \times [(a-1) \times BL) + (a \times BR)]$$

Sequential code

```
1  #include <opencv2/opencv.hpp>
2  #include <opencv2/imgproc.hpp>
3  #include <opencv2/core.hpp>
4  #include <iostream>
5  #include <filesystem>
6  #include <chrono>
7
8  using namespace cv;
9  using namespace std;
10 namespace fs = std::filesystem;
11
12
13 Mat apply_clahe(const Mat& img) {
14     Mat gray;
15     cvtColor(img, gray, COLOR_BGR2GRAY);
16
17     Ptr<CLAHE> clahe = createCLAHE(2.0, Size(8, 8));
18     Mat he_img;
19     clahe->apply(gray, he_img);
20
21     return he_img;
22 }
```


Kernel 1

```
1  __global__
2  void tile_histogram_kernel(const unsigned char* __restrict__ d_img, int imgW, int imgH,
3                           int pitch, int tileW, int tileH,
4                           int tilesX, int tilesY,
5                           unsigned int* d_tile_hist) {
6
7      int tile_idx = blockIdx.x;
8      if (tile_idx >= tilesX*tilesY) return;
9
10     int tx = tile_idx % tilesX;
11     int ty = tile_idx / tilesX;
12     int x0 = tx * tileW;
13     int y0 = ty * tileH;
14     int w = min(tileW, imgW - x0);
15     int h = min(tileH, imgH - y0);
16     int tile_pixels = w * h;
17
18     extern __shared__ unsigned int s_hist[];
19
20     for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) s_hist[i] = 0u;
21     __syncthreads();
22
23     int tid = threadIdx.x;
24     int stride = blockDim.x;
25     for (int p = tid; p < tile_pixels; p += stride) {
26         int local_x = p % w;
27         int local_y = p / w;
28         int img_x = x0 + local_x;
29         int img_y = y0 + local_y;
30         unsigned char val = d_img[img_y * pitch + img_x];
31         atomicAdd(&s_hist[val], 1u);
32     }
33     __syncthreads();
34
35     unsigned int* tile_hist = d_tile_hist + (size_t)tile_idx * NUM_BINS;
36     for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
37         tile_hist[i] = s_hist[i];
38     }
39 }
40
```


Host

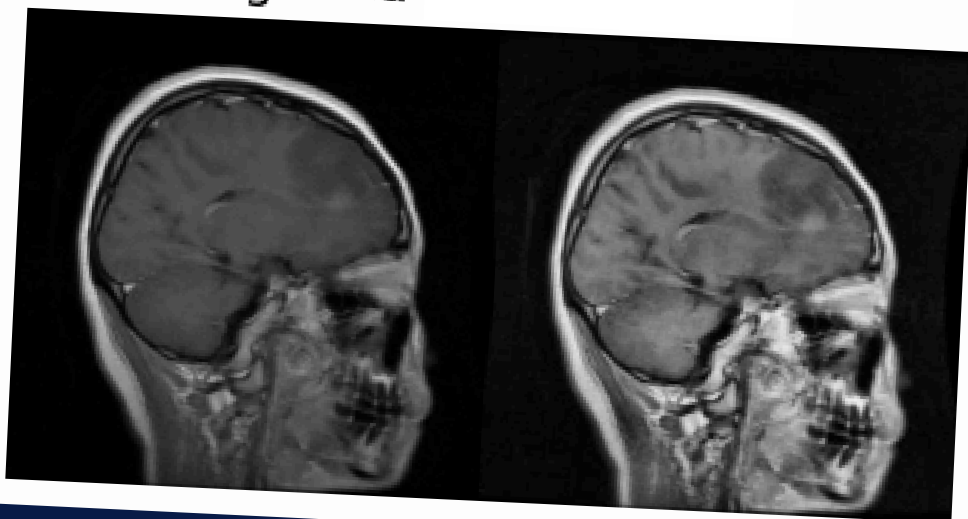
```
1 vector<unsigned char> host_all_luts((size_t)tilesCount * NUM_BINS);
2 vector<unsigned int> host_hist((size_t)NUM_BINS);
3
4 vector<unsigned int> host_all_tile_hist((size_t)tilesCount * NUM_BINS);
5 CUDA_CHECK(cudaMemcpy(host_all_tile_hist.data(), d_tile_hist, (size_t)tilesCount * NUM_BINS * sizeof(unsigned int), cudaMemcpyDeviceToHost));
6
7 for (int t = 0; t < tilesCount; ++t) {
8
9
10     for (int b = 0; b < NUM_BINS; ++b)
11         host_hist[b] = host_all_tile_hist[(size_t)t * NUM_BINS + b];
12
13
14     unsigned int excess = 0;
15     for (int b = 0; b < NUM_BINS; ++b) {
16         if (host_hist[b] > (unsigned)CLIP_LIMIT) {
17             excess += host_hist[b] - CLIP_LIMIT;
18             host_hist[b] = CLIP_LIMIT;
19         }
20     }
21
22     unsigned int add = excess / NUM_BINS;
23     unsigned int rem = excess % NUM_BINS;
24     for (int b = 0; b < NUM_BINS; ++b) host_hist[b] += add;
25     for (int b = 0; b < (int)rem; ++b) host_hist[b] += 1;
26
27     unsigned int cumsum = 0;
28     int tpixels = host_tile_pixels[t];
29     for (int b = 0; b < NUM_BINS; ++b) {
30         cumsum += host_hist[b];
31         float v = (float)cumsum / (float)tpixels;
32         unsigned char mapped = (unsigned char)min(255.0f, floorf(255.0f * v + 0.5f));
33         host_all_luts[(size_t)t * NUM_BINS + b] = mapped;
34     }
35 }
36
```

Kernel 2

```
1  __global__
2  void apply_clahe_bilinear_kernel(const unsigned char* __restrict__ d_img, unsigned char* d_out,
3                                  int imgW, int imgH, int pitch, int tileW, int tileH,
4                                  int tilesX, int tilesY,
5                                  const unsigned char* __restrict__ d_tile_lut) {
6      int px = blockIdx.x * blockDim.x + threadIdx.x;
7      int py = blockIdx.y * blockDim.y + threadIdx.y;
8      if (px >= imgW || py >= imgH) return;
9
10     float fx = (px + 0.5f) / tileW - 0.5f;
11     float fy = (py + 0.5f) / tileH - 0.5f;
12     int tx = floorf(fx);
13     int ty = floorf(fy);
14     float wx = fx - tx;
15     float wy = fy - ty;
16
17     int tx0 = min(max(tx, 0), tilesX - 1);
18     int ty0 = min(max(ty, 0), tilesY - 1);
19     int tx1 = min(max(tx + 1, 0), tilesX - 1);
20     int ty1 = min(max(ty + 1, 0), tilesY - 1);
21
22     int tile00 = ty0 * tilesX + tx0;
23     int tile10 = ty0 * tilesX + tx1;
24     int tile01 = ty1 * tilesX + tx0;
25     int tile11 = ty1 * tilesX + tx1;
26
27     unsigned char v = d_img[py * pitch + px];
28
29     unsigned char a00 = d_tile_lut[tile00 * NUM_BINS + v];
30     unsigned char a10 = d_tile_lut[tile10 * NUM_BINS + v];
31     unsigned char a01 = d_tile_lut[tile01 * NUM_BINS + v];
32     unsigned char a11 = d_tile_lut[tile11 * NUM_BINS + v];
33
34     float v0 = (1.0f - wx) * a00 + wx * a10;
35     float v1 = (1.0f - wx) * a01 + wx * a11;
36     float vmap = (1.0f - wy) * v0 + wy * v1;
37     unsigned char outv = (unsigned char)min(255.0f, max(0.0f, floorf(vmap + 0.5f)));
38     d_out[py * pitch + px] = outv;
39 }
40
```

Results

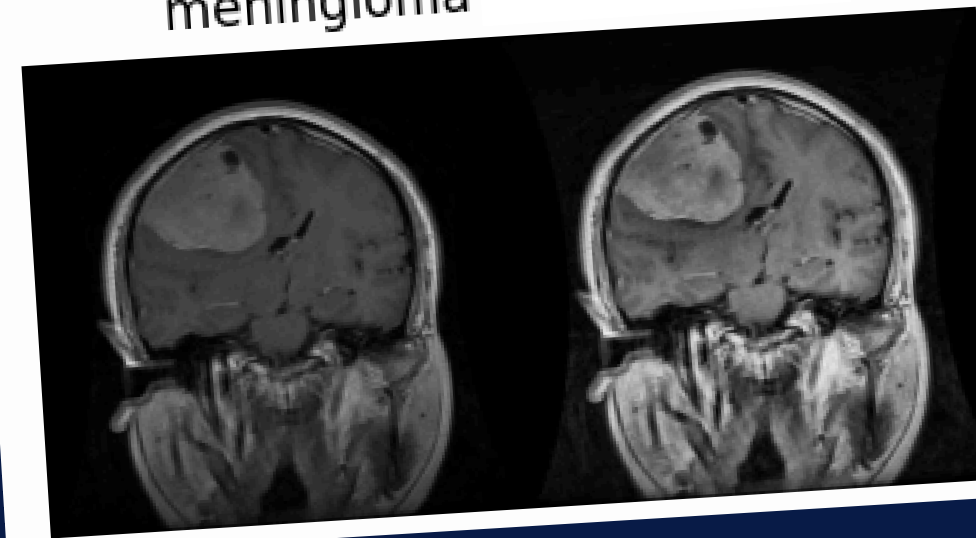
glioma



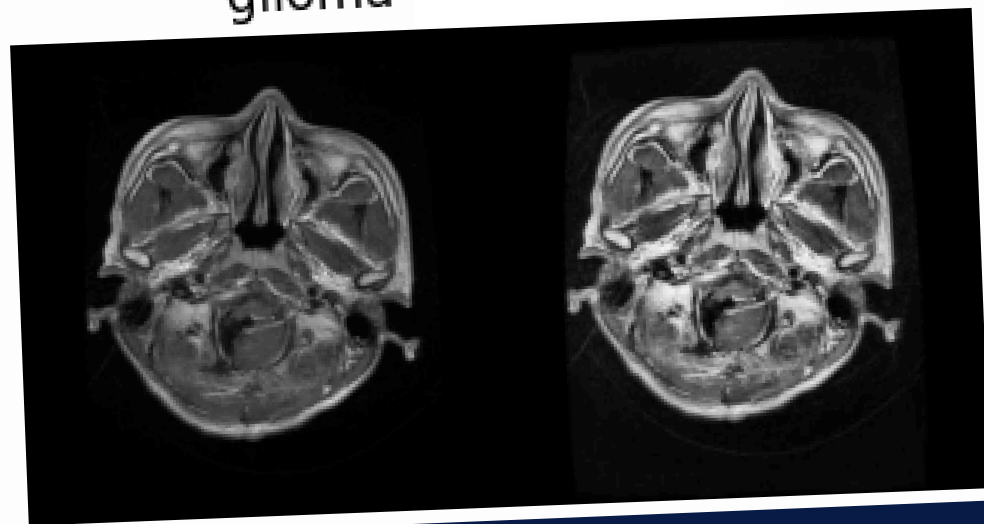
notumor



meningioma



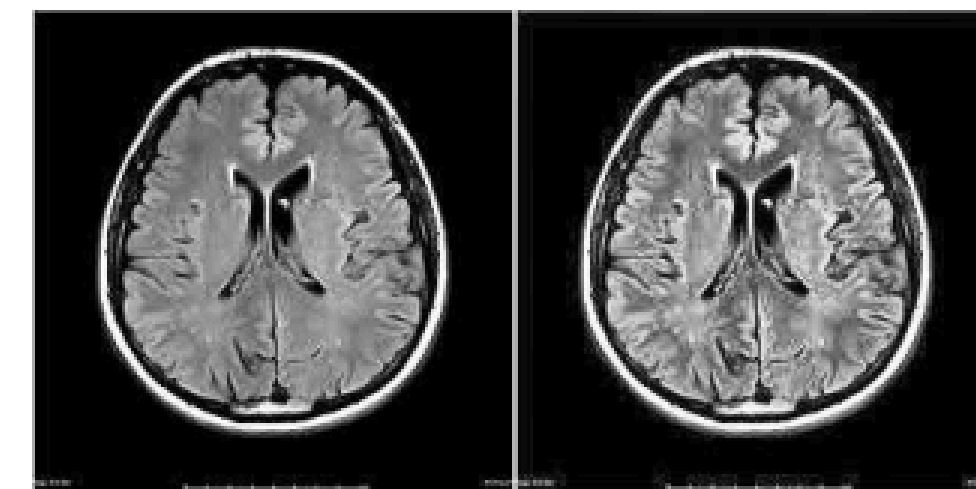
glioma



pituitary



notumor



Performance

CPU

- 703.99 seconds
- 5,712 pictures

```
Processing glioma: 100%|████████████████████| 1321/1321 [03:08<00:00, 7.00it/s]
Processing meningioma: 100%|██████████████████| 1339/1339 [02:57<00:00, 7.54it/s]
Processing notumor: 100%|███████████████████| 1595/1595 [02:10<00:00, 12.18it/s]
Processing pituitary: 100%|██████████████████| 1457/1457 [03:26<00:00, 7.05it/s]
Sequential CPU HE Done in 703.99 seconds
```



GPU

- 29.46 seconds
- 5,712 pictures



```
Processing notumor: [████████████████████████████████████████] 100% (199.38 it/s)
Processing glioma: [████████████████████████████████████████] 100% (220.17 it/s)
Processing pituitary: [██████████████████████████████████████] 100% (208.14 it/s)
Processing meningioma: [██████████████████████████████████████] 100% (223.17 it/s)

✅ ALL DONE in 29.46 seconds
```